



## 1. Problem Title & Link

**Title:** LeetCode 523 : Continuous Subarray Sum

**Link:** [LeetCode 523 - Continuous Subarray Sum](#)

## 2. Problem Statement (Short Summary)

You are given:

nums[]

k

Return true if there exists a **continuous subarray** of size at least 2 whose sum is a multiple of k.

Condition:

$\text{subarraySum} \% k == 0$

In simple terms:

Find whether any consecutive part of the array has a sum divisible by k.

## 3. Examples (Input → Output)

### Example 1

Input:

nums=[23,2,4,6,7]

k=6

Output:

true

Explanation:

Subarray:

[2,4]

Sum:

$2+4=6$

$6\%6=0$

### Example 2

Input:

nums=[23,2,6,4,7]

k=6

Output:

true

Explanation:



Subarray:

[23,2,6,4,7]

Sum:

42

$42 \% 6 = 0$

### Example 3

Input:

nums=[23,2,6,4,7]

k=13

Output:

false

### 4. Constraints

$1 \leq \text{nums.length} \leq 10^5$

$0 \leq \text{nums}[i] \leq 10^9$

$1 \leq k \leq 2^{31}-1$

Special restrictions:

- Subarray size must be at least 2
- Need better than  $O(n^2)$

### 5. Core Concept (Pattern / Topic)

**Prefix Sum + HashMap**

### 6. Thought Process (Step-by-Step Explanation)

#### Brute Force

For every index:

1. Generate all subarrays
2. Compute sum
3. Check:

$\text{sum} \% k == 0$

Time:

$O(n^2)$

Very slow.

#### Optimized Thinking



Observation:

If:

$\text{prefixSum1} \% k$

$==$

$\text{prefixSum2} \% k$

Then:

$(\text{prefixSum2} - \text{prefixSum1})$

$\%k == 0$

Meaning:

Subarray between them is divisible by  $k$ .

Algorithm:

1. Compute running prefix sum
2. Store remainder:

$\text{prefixSum} \% k$

in HashMap

3. If same remainder appears again:
  - Check subarray size  $\geq 2$
  - Return true

## 7. Visual / Intuition Diagram (ASCII Diagram)

Input:

$\text{nums} = [23, 2, 4, 6, 7]$

$k = 6$

Prefix sums:

$23 \rightarrow 5$

$25 \rightarrow 1$

$29 \rightarrow 5$

Notice:

$\text{remainder} = 5$

appears again.

Subarray:



[2,4]

Sum:

6

Divisible by:

6

### 8. Pseudocode (Language Independent)

```
map={0:-1}
```

```
prefix=0
```

```
for i in range(n):
```

```
    prefix+=nums[i]
```

```
    remainder=prefix%k
```

```
    if remainder exists:
```

```
        if i-map[remainder]>=2:
```

```
            return true
```

```
    else:
```

```
        map[remainder]=i
```

```
return false
```

### 9. Code Implementation

#### Python

```
class Solution:
```

```
    def checkSubarraySum(self, nums, k):
```

```
        remainderMap = {0:-1}
```



```
prefix = 0

for i in range(len(nums)):

    prefix += nums[i]

    remainder = prefix % k

    if remainder in remainderMap:

        if i - remainderMap[
            remainder
        ] >= 2:

            return True

    else:

        remainderMap[
            remainder
        ] = i

return False
```

## Java

```
class Solution {

    public boolean checkSubarraySum(
        int[] nums,
        int k) {

        Map<Integer,Integer>
            map=new HashMap<>();

        map.put(0,-1);

        int prefix=0;
```



```
for(int i=0;
    i<nums.length;
    i++){

    prefix+=nums[i];

    int remainder=
        prefix%k;

    if(map.containsKey(
        remainder)){

        if(i-
            map.get(
                remainder)
            >=2){

            return true;
        }
    }

    else{

        map.put(
            remainder,
            i
        );
    }
}

return false;
}
```

## 10. Time & Space Complexity

Time Complexity:

$O(n)$



Reason:

Array traversed once.

Space Complexity:

$O(\min(n,k))$

Reason:

HashMap stores remainders.

### 11. Common Mistakes / Edge Cases

1. Forgetting initial value:

map={0:-1}

Without this:

Subarrays starting from index 0 fail.

2. Ignoring subarray length requirement

Wrong:

length=1

Need:

length>=2

3. Updating existing remainder

Wrong:

map[remainder]=i

Keep first occurrence only.

4. Missing:

[0,0]

Example.

### 12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[23,2,4,6,7]

k=6

| i | nums[i] | Prefix Sum | Remainder | Map            |
|---|---------|------------|-----------|----------------|
| 0 | 23      | 23         | 5         | {0:-1,5:0}     |
| 1 | 2       | 25         | 1         | {0:-1,5:0,1:1} |



|   |   |    |   |       |
|---|---|----|---|-------|
| 2 | 4 | 29 | 5 | Found |
|---|---|----|---|-------|

Check:

$$2 - 0 = 2$$

Valid length.

Return:

true

### 13. Common Use Cases (Real-Life / Interview)

Financial analysis

Load balancing systems

Periodic event detection

Signal processing

### 14. Common Traps (Important!)

1. Overwriting first occurrence in map
2. Missing initial:

0:-1

3. Using nested loops
4. Ignoring subarray size requirement

### 15. Builds To (Related LeetCode Problems)

Progression:

- LC 523 → Continuous Subarray Sum
- LC 560 → Subarray Sum Equals K
- LC 974 → Subarray Sums Divisible by K
- LC 325 → Maximum Size Subarray Sum Equals K
- LC 930 → Binary Subarrays With Sum

### 16. Alternate Approaches + Comparison

| Approach             | Time     | Space           | Notes           |
|----------------------|----------|-----------------|-----------------|
| Brute Force          | $O(n^2)$ | $O(1)$          | Too slow        |
| Prefix Sum Array     | $O(n^2)$ | $O(n)$          | Still expensive |
| Prefix Sum + HashMap | $O(n)$   | $O(\min(n, k))$ | Optimal         |





### 17. Why This Solution Works (Short Intuition)

If two prefix sums leave the same remainder when divided by  $k$ , their difference must be divisible by  $k$ . That difference represents a subarray sum, allowing us to detect valid subarrays efficiently.

### 18. Variations / Follow-Up Questions

1. Return subarray indices
2. Count all valid subarrays
3. Find longest valid subarray
4. Solve with negative numbers
5. Find subarrays divisible by multiple values
- 6.